

Permisos por rol en los menús

Guía para reproducir manualmente los cambios del frontend en otra rama

Proyecto: Sistema de Legajos — Bomberos Voluntarios

Rama analizada: feature/frontend-infraestructura

Commit de referencia: 6275b4f — implementación de RBAC con sessionStorage

Alcance: interfaz React. No modifica ni protege el backend.

1. Objetivo y funcionamiento

El objetivo es que React muestre, oculte o deshabilite opciones según el rol del usuario. El flujo implementado en la rama de referencia es:

1. El login obtiene o genera un token con un rol.
2. El token se guarda en `sessionStorage`.
3. `AuthContext` decodifica el token y publica el rol para toda la aplicación.
4. `Navbar` entrega el rol a los menús de escritorio y móvil.
5. `hasPermission()` decide qué elementos se renderizan.

Rol	Permisos de la implementación actual
ROLE_ADMIN	crear, editar, eliminar, leer
ROLE_INSTRUCTOR	editar_notas, leer
ROLE_USER	leer

Importante: ocultar un menú no protege la API. Un usuario todavía podría llamar al backend directamente. Esta guía reproduce la capa visual; la autorización definitiva debe realizarse también en Flask.

2. Archivos involucrados

Archivo	Responsabilidad
<code>src/utils/authHelper.js</code>	Matriz de permisos y función <code>hasPermission</code> .
<code>src/utils/auth.js</code>	Decodificación del JWT y normalización del rol.
<code>src/context/AuthContext.jsx</code>	Estado global del rol, login y logout.
<code>src/App.jsx</code>	Coloca <code>AuthProvider</code> alrededor de la aplicación.
<code>src/pages/Login.jsx</code>	Entrega el token a <code>login()</code> .
<code>src/components/Navbar.jsx</code>	Obtiene el rol y lo pasa a ambos menús.
<code>src/components/DesktopNav.jsx</code>	Filtra opciones del menú de escritorio.
<code>src/components/MobileDrawer.jsx</code>	Filtra opciones del menú móvil.

3. Paso 1 — Matriz de permisos

Crear frontend/src/utils/authHelper.js:

```
export const PERMISSIONS_MAP = {
  ROLE_ADMIN: ["crear", "editar", "eliminar", "leer"],
  ROLE_INSTRUCTOR: ["editar_notas", "leer"],
  ROLE_USER: ["leer"],
};

export function hasPermission(userRole, action) {
  if (!userRole) return false;
  const permissions = PERMISSIONS_MAP[userRole] || [];
  return permissions.includes(action);
}
```

Ejemplos:

```
hasPermission("ROLE_ADMIN", "crear");    // true
hasPermission("ROLE_USER", "crear");     // false
hasPermission("ROLE_INSTRUCTOR", "leer"); // true
```

Esta matriz utiliza permisos genéricos. Una versión más precisa debería utilizar permisos por recurso, por ejemplo `personas.leer`, `planes.crear` y `configuracion.editar`.

4. Paso 2 — Leer el rol del token

Crear frontend/src/utils/auth.js. Esta versión decodifica el contenido del JWT; no verifica su firma:

```
export function parseJWT(token) {
  try {
    const payload = token.split(".")[1];
    const normalized = payload.replace(/-/g, "+").replace(/_/g, "/");
    const decoded = decodeURIComponent(
      atob(normalized)
        .split("")
        .map((char) => "%" + ("00" + char.charCodeAt(0).toString(16)).slice(-2))
        .join("")
    );
    return JSON.parse(decoded);
  } catch {
    return null;
  }
}

export function obtenerRolNormalizado(payload) {
  if (!payload) return null;
  const rawRole = payload.role || payload.rol || payload.authorities;
  const role = Array.isArray(rawRole) ? rawRole[0] : rawRole;
  const value = String(role || "").toUpperCase();

  if (value.includes("ADMIN")) return "ROLE_ADMIN";
  if (value.includes("INSTRUCTOR")) return "ROLE_INSTRUCTOR";
  return "ROLE_USER";
}
```

5. Paso 3 — Contexto de autenticación

Crear frontend/src/context/AuthContext.jsx:

```
import { createContext, useContext, useState } from "react";
import { parseJWT, obtenerRolNormalizado } from "../utils/auth";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [currentUserRole, setCurrentUserRole] = useState(() => {
    const token = sessionStorage.getItem("token");
    return token ? obtenerRolNormalizado(parseJWT(token)) : null;
  });

  function login(token) {
    sessionStorage.setItem("token", token);
    setCurrentUserRole(obtenerRolNormalizado(parseJWT(token)));
  }

  function logout() {
    sessionStorage.removeItem("token");
    setCurrentUserRole(null);
  }

  return (
    <AuthContext.Provider value={{ currentUserRole, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  return useContext(AuthContext);
}
```

Envolver la aplicación

En App.jsx, importar el proveedor y envolver el contenido:

```
import { AuthProvider } from "../context/AuthContext";

function App() {
  return (
    <AuthProvider>
      {/* Toaster, Routes y demás contenido */}
    </AuthProvider>
  );
}
```

6. Paso 4 — Conectar el login

El componente de login debe obtener login desde el contexto:

```
import { useAuth } from "../context/AuthContext";

function InicioSesion() {
  const { login } = useAuth();

  async function ingresar(event) {
    event.preventDefault();

    // En producción, este token debe venir del backend.
    const respuesta = await autenticar(usuario, password);
    login(respuesta.token);
    navigate("/inicio");
  }
}
```

La rama analizada genera un JWT simulado y decide el rol mirando si el usuario contiene “admin” o “instructor”. Eso sirve únicamente para demostrar la interfaz. No debe copiarse como autenticación real.

7. Paso 5 — Pasar el rol al Navbar

En Navbar.jsx:

```
import { useAuth } from "../context/AuthContext";

function Navbar() {
  const { currentUserRole, logout } = useAuth();

  return (
    <>
      <DesktopNav currentUserRole={currentUserRole} />
      <MobileDrawer currentUserRole={currentUserRole} />
    </>
  );
}
```

8. Paso 6 — Filtrar el menú de escritorio

En DesktopNav.jsx:

```
import { hasPermission } from "../utils/authHelper";

export default function DesktopNav({ currentUserRole }) {
  return (
    <nav>
      {hasPermission(currentUserRole, "leer") && (
        <button onClick={() => navigate("/config-documentos")}>
          Configuración / Catálogos
        </button>
      )}

      {hasPermission(currentUserRole, "crear") && (
        <button onClick={() => navigate("/crearLegajo")}>
          Nuevo Legajo
        </button>
      )}
    </nav>
  );
}
```

El mismo patrón se repite para Planes, Personas, Asignaturas y cualquier otra opción:

```
{hasPermission(currentUserRole, "crear") && (
  <button onClick={() => navigate("/planes/alta")}>
    Nuevo Plan de Estudio
  </button>
)}
```

9. Paso 7 — Repetirlo en móvil

En `MobileDrawer.jsx` se utiliza la misma matriz:

```
import { hasPermission } from "../utils/authHelper";

{hasPermission(currentUserRole, "crear") && (
  <button onClick={() => navigate("/alta-persona")}>
    Registrar Persona
  </button>
)}
```

Es importante actualizar ambos componentes. Si solo se modifica `DesktopNav`, las opciones continuarían visibles desde un teléfono.

10. Recomendación: permisos por recurso

Para que los roles controlen realmente qué secciones aparecen, conviene reemplazar los permisos genéricos por permisos específicos:

```
export const PERMISSIONS_MAP = {
  ROLE_ADMIN: ["*"],

  ROLE_INSTRUCTOR: [
    "personas.leer",
    "planes.leer",
    "asignaturas.leer",
    "notas.editar",
  ],

  ROLE_USER: [
    "personas.leer",
  ],
};

export function hasPermission(role, permission) {
  const permissions = PERMISSIONS_MAP[role] || [];
  return permissions.includes("*") || permissions.includes(permission);
}
```

Entonces cada menú puede decidirse de forma independiente:

```
{hasPermission(currentUserRole, "personas.leer") && (
  <button onClick={() => navigate("/personas")}>Personas</button>
)}

{hasPermission(currentUserRole, "planes.crear") && (
  <button onClick={() => navigate("/planes/alta")}>Nuevo plan</button>
)}
```

11. Pruebas manuales

Caso	Resultado esperado
Sin sesión	No deberían mostrarse opciones privadas.
ROLE_ADMIN	Ve lectura, altas, edición y eliminación.
ROLE_INSTRUCTOR	Ve consultas y la edición específica de notas.
ROLE_USER	Ve solamente las opciones de lectura autorizadas.
Cerrar sesión	Se elimina el token y desaparece el rol.
Vista móvil	Respetar exactamente los mismos permisos que escritorio.
Recargar página	El rol se recupera desde <code>sessionStorage</code> .

12. Lista de verificación

- Crear `authHelper.js` y definir roles.
- Crear `auth.js` para extraer el rol.
- Crear `AuthContext.jsx`.
- Envolver la aplicación con `AuthProvider`.
- Conectar el token del login con `login(token)`.
- Entregar `currentUserRole` al `Navbar`.
- Filtrar `DesktopNav`.
- Filtrar `MobileDrawer`.
- Probar todos los roles en escritorio y móvil.
- No utilizar el token simulado en producción.
- Agregar posteriormente protección de rutas React.
- Agregar posteriormente autorización en los endpoints Flask.

Resultado esperado: cada usuario ve solamente las opciones de navegación correspondientes a su rol. La protección del backend debe implementarse como una etapa separada y obligatoria.

Documento generado a partir del análisis en modo lectura de la rama `feature/frontend-infraestructura`. No se realizaron cambios funcionales en el frontend.